

第二大脑系统架构总结

运转机制 · 核心亮点 · 真实缺陷

文档性质	系统框架总结与缺陷评估
描述口径	只涉及整体架构，不涉及具体项目
数据基准	系统自述的最近一次健康检测快照
数据来源	系统自身度量产物与运行日志
脱敏状态	路径 / 标识 / 设备 / 个人信息均已剔除

本文所有数字均为系统自身产物的原值，未作估算或修饰。

目录

1 系统定位	2
2 分层架构	2
2.1 内容层的五种材料	2
2.2 状态层：不可重建的部分	3
2.3 工具层的命令面	3
3 运转机制：用户实际怎么用	4
3.1 一个会话的完整生命周期	4
3.2 用户实际输入的东西	4
3.3 工作区天然交接	4
3.4 焦点与任务的分离	5
4 亮点一：多 Agent 协作	5
4.1 共享方式：薄入口，而非复制规则	5
4.2 跨 Agent 独立复现的设计意图（注意：此机制未落地，见 §9.1）	5
4.3 协作的产物是分工，而非全员同工	5
5 亮点二：知识沉淀闭环	6
5.1 五级来源等级	6
5.2 完整沉淀链路	6
5.3 适用范围是一等公民	6
6 亮点三：雷区避雷	7
6.1 雷区的三档生命周期	7
6.2 观察位与回报义务	7
7 亮点四：上下文 token 耗费低	7
7.1 启动包预算	7
7.2 分层记忆的体积上限	8
7.3 节省 token 的四个机制	8
8 亮点五：执行层的强制力	8
9 真实缺陷	9
9.1 若干被文档承诺、但实现层并不存在的机制（最严重）	9
9.2 维护链在报错中运行，提炼积压巨大	9
9.3 学习闭环事实上很冷	10
9.4 消费链积压，末端必须人工	10
9.5 数据安全存在双重缺口	10
9.6 状态层在仓库外，备份覆盖未核实	10

9.7 检索能力的局限	11
9.8 Hook 强制力依宿主而异	11
9.9 其余落差（择要）	11
10 缺陷的结构归因	11
11 总结评价	12
11.1 做对了什么	12
11.2 真实的短板	12
11.3 一句话概括	13
附录：数据来源	13

1 系统定位

这是一套面向多个 AI Agent 的共用长期记忆系统。它的核心命题不是「给某个 AI 加一个知识库」，而是解决四个具体问题：

问题	传统做法的失效点	本系统的应对
每次新会话从零开始	上下文窗口关闭即遗忘	会话启动时自动注入压缩后的记忆包
多个 AI 之间互不知情	各自为政，同一个坑重复踩	单一共享仓库 + 跨 Agent 独立验证
说过的偏好反复重申	偏好散落在历史对话里	偏好升格为受治理的正式条目
Agent 自以为做完	无回执、无版本、无审计	写入登记 + 版本号 + 幂等日志

系统有一条设计前提被明确写在接入指南里，值得单独引用，因为它是整套架构所有取舍的源头：

设计第一原则

任何依赖 Agent 会话中途自觉、或依赖用户主动发起的环节，等于不存在。

这条前提带来一个可验证的推论：**必读内容必须放在会话开始的固定动作里**，而不能指望「中途想起来」。整套渐进式加载、启动包、Hook 检查都是为了满足这条约束。

2 分层架构

系统在纵向上分为**四层**，职责与可重建性严格区分。

层	职责
执行 / 治理层	Hook 策略与宿主适配器；五个固定检查点；硬阻断能力依宿主而异
工具层	Python CLI（二十余个子命令）：启动包、检索、保存、交接、提炼、发布、巡检
状态 / 账本层	不可重建 ：候选队列、操作日志、版本登记、来源快照、费用账本
内容层	版本化 Markdown 仓库：权威事实、证据、派生、候选、归档五种材料

2.1 内容层的五种材料

内容层按**可信度与生命周期**分档，而不是按主题分档。

档位	性质	是否权威	新窗口是否默认读取
正式事实	习惯、知识、雷区、技能、核心热记忆	权威（须经发布链写入）	仅压缩投影进启动包
证据	会话总结、跨 Agent 日志	只证明「当时做过」	否，按需追溯
派生	记忆包、健康度量、各类索引	可随时重建	记忆包必读
候选	收件箱、候选队列	未经确认，不得当事实	否
归档	已替代、已撤出材料	仅作历史去向	否，默认不检索

权威与派生副本的边界被显式声明：系统维护一份「权威源 → 派生副本」的关系清单，并规定工具只报告「发现不一致」，**不判断谁对**。这是一个刻意的克制——避免工具自动「修正」出一个可能是错的版本。

2.2 状态层：不可重建的部分

这是整套系统里最容易被误解、也最需要保护的部分。

- 索引可重建 ≠ 账本可丢弃。检索索引能从现有 Markdown 重新切分出来，但生成不出候选来源、操作意图、版本历史、来源快照和已消费预算。
- 因此候选队列、操作日志、版本登记（写命令的并发控制依据）、来源快照、反馈状态、费用账本被归为**不可重建**。
- 而**状态根**位于仓库之外，仓库的备份策略**不自动覆盖它**。系统自己把这一点标记为「当前最大的单点风险」。

2.3 工具层的命令面

工具层是一个统一的 CLI，命令按「是否写盘」和「是否需要人工授权」分级。

类别	代表命令	默认行为
只读诊断	巡检诊断、写入登记对账、宿主接入检查、	只读，不跑模型、不改数据状态冻结
读 + 绑定	启动包、有界检索	只读，输出任务绑定
预览型写	索引构建、候选入队、正式发布	默认只预览，须显式确认才真写
阶段保存	检查点、交接、单次语义提交	携带期望版本号，冲突即拒绝
长时作业	日常增量维护、反馈消费、历史提炼	受预算与时间窗约束
事实变更	正式事实变更发布通道	独立通道，可回滚

设计上做了两个防御：

1. 双闸门——所有写命令默认预览，必须显式加写入开关才真写。这一条从机制上避免了「以为在看，其实在写」。
2. 演练模式下不可能产生计费——干跑时预留一律被拒绝。

3 运转机制：用户实际怎么用

3.1 一个会话的完整生命周期

一个会话的完整生命周期

宿主自动读取薄入口（引导文件，不含规则正文）

Hook 检查入口文件存在性 + 记忆包新鲜度

只检查，不声称已把正文交给模型

执行一次启动引导命令，返回：约束块 + 任务身份 + 当前目标/下一步 + 适用线索

【不读】 全量规则、首页全文、所有项目状态、完整共享包

干活（需要历史时才发起有界检索，默认最多 5 条）

有效阶段结束时保存一次（检查点 / 交接）

携带期望版本号 · 核对回执与读回版本

会话结束：写会话总结；有真实新收获才写候选块

3.2 用户实际输入的东西

用户的日常输入量被压到很低，这是整套设计的目标之一。

时机	用户 / Agent 动作	频率
新窗口	一条引导命令（含工作区与宿主名）	每窗口一次
同窗口再问	加「已知规则版本」参数	版本未变则零成本
需要历史	一条有界检索命令	按需
阶段完成	一条保存命令	每有效阶段一次
换窗口	一条交接命令	交接时

关键点：用户不需要在每次会话里重复申明偏好、背景和约束——这些由启动包在会话最开始一次性注入。

3.3 工作区天然交接

交接不是「把对话记录复制过去」，而是把任务身份和现场结构化地移交。

- 每个任务绑定一个三元组：（项目标识，任务标识，交接卡路径）。
- 保存时携带这套绑定，因此不会被其他窗口改变焦点后换错卡。
- 交接卡带修订号；新窗口读回时必须核对版本。
- 冲突时拒绝覆盖并给出当前哈希，要求显式确认接管。

- 重复提交同一操作标识 → **幂等去重**（不是丢写），回执明确说明「重复请求已去重」。
- 过期版本号写入 → **拒绝**，并报告「当前记录修订号与提交方期望值不符」。

这套机制让**任意 Agent、任意窗口、任意时间**都能接手一个任务，而不依赖「上一个窗口的对话还在」。这正是工作区自然交接的实现基础。

3.4 焦点与任务的分离

系统特意区分了两个概念：**绑定**（这个窗口属于哪个任务，保存时用）与**焦点**（新窗口默认选哪个任务，仅供默认选择）。

由此产生两条纪律：**已完成的任务卡不自动重启；只有需要改变新窗口默认任务时才动焦点。**

4 亮点一：多 Agent 协作

4.1 共享方式：薄入口，而非复制规则

多个不同类型的宿主（命令行 Agent、编辑器内 Agent、桌面应用 Agent、独立运行时）接入了同一个仓库。接入方式刻意做成了**薄入口**：

- 每个宿主的全局指令文件只做引导，**不承载规则正文**。
- 正式规则**只有一份权威**，其余入口全部指向它。
- 系统记录了历史教训：早期曾出现「两个宿主各自承载 80–115 行规则正文，与权威规则形成**三套并存的 schema**」，收敛时花了大力气返工。

由此得出一条明确原则：**薄入口越长，规则漂移越快。**

4.2 跨 Agent 独立复现的设计意图（注意：此机制未落地，见 §9.1）

系统在设计层提出了一条很有价值的思路。

按不同的 **Agent 名** 判断「有多少独立来源确认过这条经验」。

单 Agent 重复再多也升不上去——因为同一个模型说十遍错话仍是错话。

为了让这个机制有效，接入规范强制要求：**Agent 名必须唯一、稳定、大小写固定**。如果同一个 Agent 被记成两种写法，系统会把它们当成两个独立来源，从而**虚增置信度**，让未经真实验证的经验冒充「被多 Agent 确认」。

也就是说，**命名一致性不是格式洁癖**，它直接决定置信度计算的正确性。

但必须指出：经代码核查，「按独立 Agent 来源数升权」在实现里并不存在——承载它的自动计数组件已随旧采收器一同退役，而设计文档里「反复被独立确认会自动升权」的表述**没有被同步更新**。这构成了本系统最典型的一类缺陷：**机制被退役了，而承诺它的文案还留着**（详见 §9.1）。

4.3 协作的产物是分工，而非全员同工

系统里沉淀了明确的协作编排模式：由高能力模型负责拆解任务、答疑与审查，由低成本模型在独立工作区施工，只有「理解门、授权、测试、审查」全部通过才允许合并。这使多 Agent 协作从「多开几个窗口」升级为**有成本模型的分工体系**。

5 亮点二：知识沉淀闭环

5.1 五级来源等级

所有事实性内容必须带来源等级，且只有前三级允许进入事实层。

等级	含义
user_confirmed	用户明确确认
locally_verified	本地实测验证
external_trusted	可信外部来源
——以上可进事实层，以下仅作参考——	
agent_inference	Agent 推断（仅线索）
unknown	来源不明（仅线索）

配套纪律包括：派生物必须声明来源与时点；未知不能伪装已知；历史不能补写假证据或作者；修改正式事实必须经过发布链。

5.2 完整沉淀链路

会话中的真实收获 → 收获块（收件箱）→ 候选队列
↓ 语法校验 + 去重 + 作用域检查
机器复核（此段需要模型能力）→ 暂存发布清单（含审计与回滚记录）
↓ 人工策展确认
正式发布 → 索引回读（可检索、可引用）

链路里有几个刻意设置的闸门：

- 语法校验是硬闸门：格式不合法的收获块整块被拒并进入拒绝区，不会静默丢失。
- 作用域不得降级：没声明适用范围时，系统不会默认放宽为全局，而是判为「待确认」——宁可当作线索，也不冒充已验证。
- 人工策展不可省：机器只能写到「暂存发布清单」，正式发布需要人的确认。
- 反馈不自动验证知识：使用反馈只影响匹配场景的召回，不会自动把一条经验升级为已验证。

5.3 适用范围是一等公民

检索采用多维度作用域模型：全局 / 特定 Agent / 特定项目 / 特定工具 / 特定场景。

- 不同维度之间是 AND，同一维度多值之间是 OR。
- 未知范围不放宽为全局。
- 记录者不等于适用者——「谁记的」和「对谁适用」是两件必须分开声明的事。

这一条解决了很多记忆系统的通病：一条在特定工具下成立的经验，被当成普适真理到处套用。

6 亮点三：雷区避雷

6.1 雷区的三档生命周期

失败经验不是简单入库就生效，而是走一条**观察期** → **生效** → **退役**的路径。

档位	含义	是否注入新会话
观察档	已入库，证据强度待验证	不进提醒，进入观察位轮转
生效档	已验证，具备提醒资格	进提醒 （按置信度 × 严重度排序）
退役档	已过期或被替代	不计入提醒，仅作历史

6.2 观察位与回报义务

系统不把「待验证」藏着，而是**主动暴露并索取反馈**。

- 每次会话从观察档里轮转**固定数量**的条目注入（轮转指针可见）。
- 被注入的条目附带**明确要求**：本次会话如果实际涉及对应工具，**必须回报结果**。
- 回报有四种标准取值：**avoided**（规避成功）/ **hit_anyway**（仍然踩中）/ **falsified**（被证伪）/ **irrelevant**（不相关）。
- **命中与证伪都能改进系统**——证伪同样是有价值的信号。

这是一个**闭环设计**：不只是「告诉 Agent 别踩坑」，还回收「这次到底踩没踩」，从而让置信度不是纸面推测。

7 亮点四：上下文 token 耗费低

这是整套系统在工程上最扎实的部分，有多个**可核验的数字约束**。

7.1 启动包预算

项目	值
默认预算	1800 token
可选档位	精简 1800 / 标准 3200 / 审计 5000
裁剪规则	关键块（规则约束、任务卡、当前风险） 全保留 ；可选块按优先级丢弃
超预算行为	显式报告超支，不静默丢弃关键内容

有一个设计决策值得单独记录：评审**否决了「直接上调默认预算」**，理由是「那会把启动包设计问题变成预算问题」。同时**明确不做自动升档**，因为自动升档会让超支**静默消失**，与「检测失败必须暴露」的原则冲突。

7.2 分层记忆的体积上限

层	上限	注入方式
核心热记忆	$\leq 2 \text{ KB}$	无条件注入（只放「半年不变级」环境事实）
上下文包	$\leq 8 \text{ KB}$ （实测约 6.6 KB）	会话开始必读，单段式注入，不二次读取
检索结果	默认最多 5 条	只在需要时发起

核心热记忆有明确的内容纪律：用户偏好一律以习惯库为准，本文件不复制；失败模式进雷区库，不重复。也就是说体积上限不仅靠「删」，更靠职责不重叠。

7.3 节省 token 的四个机制

1. 同窗口版本缓存——规则版本未变则只返回一行版本确认，不重复注入规则正文。
2. 渐进式阅读——导航顺序是从概览到具体，信息足够且风险低即停；只有不足、冲突、过期或高风险时才深入。禁止默认全库扫描（Hook 会对递归整库扫描给出提醒）。
3. 有界检索——最多 5 条，最多一次改写，一次改写仍无答案就说明边界。信息够用即停，不等待后台学习。
4. 派生物重建而非持久化——派生目录整体被版本控制排除，随时可重建，不需要人工维护一致性。

综合效果：一个全新窗口的启动上下文成本被压到 1–2 千 token 量级，同时仍能拿到任务身份、当前目标、适用偏好与风险提醒。

8 亮点五：执行层的强制力

薄入口负责「告诉 Agent 应该怎么做」，Hook 负责「在关键时点自动检查或阻断」。系统明确声明 Hook 只是执行层，不得另造一套规则正文。它只实现五项检查，不是通用安全框架。

检查点	行为
会话启动	检查入口文件与记忆包新鲜度；过期或条目为零时提示失效
检索范围	发现默认全库递归扫描时给出提示
写入分级	正式层写入硬拒绝；只放行候选区、附件区与本 Agent 总结区
总结触发	第一次成功状态变化后建立并复用一份总结；失败的工具调用不触发
文档超限	项目文档超过 20 KB 且继续增长时只提醒，不阻止、不回滚

三个工程上成熟的做法：

1. 硬软分级——硬 Hook（写入分级、凭据读取、禁止整体覆盖）在宿主支持时直接拒绝，且自身失败时优先 fail-closed；软 Hook（扫描提醒、文档超限）绝不回滚已经成功的操作。

2. 不谎报能力——「宿主不支持阻断」或「没有本机可执行宿主做真实烟测」时，只能标记为审计 / 兼容模式，不能宣称已形成硬约束。
3. 诚实的能力边界声明——文档明确写着：正则属于提醒和防误操作，不是安全沙箱；异步任务完成后 Agent 必须自行确认并保存。

9 真实缺陷

以下全部来自系统自身的度量产物与运行日志，不含推测。按严重度排列。

9.1 若干被文档承诺、但实现层并不存在的机制（最严重）

机制已退役或从未实现，而承诺它的文案仍在生效。

被承诺的机制	实况
按独立 Agent 来源数自动升权	不存在——计数组件已随旧采收器退役
五级来源等级	基本未实现，该字段无枚举校验
候选 → 已验证路径	无代码路径：机器复核明文「不授予已验证」
候选队列正式晋升	从未发生：账本 728 行，晋升事件 0 次
付费预算软硬件限	已实现但无生产调用点，护栏是惰性的

根源：自动化沉淀链存在两条并行的候选来源，从未缝合——一条走「收件箱 → 候选队列」，另一条走「历史提炼 → 暂存区 → 机器复核」。工具自己承认：「两个候选源，晋升协议断裂。」

即「知识沉淀闭环」账面上是闭环、运行上是断链。§5 应读作设计意图，不是已跑通的现状。

9.2 维护链在报错中运行，提炼积压巨大

指标	值
定时维护作业	最后运行结果为失败（返回码非零）
提炼产物	无法解析，判定为不可恢复（JSON 解析失败）
本轮提炼成功数	0
待处理来源队列	约 1813（扫描到 1902）
单轮实际尝试	2（配置上限：2 来源 / 240 秒 / 1 块）

根因：维护链依赖一个本地小参数模型做结构化提炼，而把任意模型输出可靠约束成合法 JSON 是固有薄弱点。失败模式不是「偶尔慢」，而是整轮归零。按当前吞吐，清空积压需约 900 轮以上——这是结构性不匹配，非偶发延迟。

9.3 学习闭环事实上很冷

指标	实测值	对照
雷区条目总数	484	—
生效档条目	5（占 1.0%）	—
观察档池	455（多数「观察 0 次」）	—
从未收到任何反馈的条目	98.0%	学术基线 44.9%
反馈中「仍然踩中」/「被证伪」	各 0 条	—

系统的结论原文：「观察位加大探索买不到归因」。直白说：雷区库看似很大，但真正能起提醒作用的只有 5 条；98% 的条目从未产生过一次有效性反馈——置信度体系缺乏真实数据支撑。

这是「知识沉淀」与「知识可用」之间的真实鸿沟：入库量不等于可用量。

9.4 消费链积压，末端必须人工

指标	值
待审收获块	66–67（最老积压 8 天）
候选：重复	193
候选：因作用域未声明被阻塞	4
正式晋升	0

系统自己的免责声明：「写入成功 ≠ 经验已进入可用记忆。计数只表示材料在场。」

链路末端（正式晋升）必须有模型能力且必须人工策展确认——无法完全自动化，必然成为瓶颈。

9.5 数据安全存在双重缺口

- 版本控制未收口：未提交变更 3000+ 项，最近一次提交在数日前，之后的改动全部只存在于磁盘上。
- 快照目录为空：实测 0 文件，且是「从未产生」还是「已被清理」未确定——因此回滚能力只是能力判断，不是损失评估。

两者叠加，意味着没有可回退的基线。

9.6 状态层在仓库外，备份覆盖未核实

状态根（含费用账本、日志）位于仓库之外，系统自述「仓库备份策略不自动覆盖它」，并把此项列为「当前最大的单点风险」。

9.7 检索能力的局限

- 索引约 1187 文件 / 19122 分节，检索为关键词 + 分节，无语义能力；「最多 5 条」叠加无向量召回，同义改写易漏检。
- 无真正增量：索引是整库重建，「增量」实为按时间戳触发全量重建，成本随规模线性增长。
- 两套索引并存：同用途索引维护了两份（约 102 MiB + 118 MiB）。
- 边界声明不一致：「最多 5 条」有代码强制，「最多一次改写」没有；启动包内部会发起 5–10 次检索，且静默吞掉内层告警——只会「少一块内容」，看不到「为什么少」。

9.8 Hook 强制力依宿主而异

硬阻断只在支持相应拦截点的宿主上可靠；多个宿主的接入被标注为「兼容 / 审计模式」（无本机可执行宿主做真实阻断烟测）。「写入治理」目前是分级能力，不是统一保证。

此外，真实多宿主接力从未成功过一次：静态检查全部通过，而真实窗口启动全部失败。系统表述克制——「零条真实接力，不伪造成功」。这是诚实的失败，但也说明该架构「在测试里成立，在真机上未被验证」。

9.9 其余落差（择要）

- 雷区注入实际为 0 条：484 条中 479 条被过滤，455 条卡在已退役的观察档。
- 坏消息的通知通道是空的：采收器停摆 8 天，而其告警文件全库 0 命中（从未存在）。
- 同一指标多处对不上：至少 5 处口径漂移（雷区数相差 74、拒绝数登记 11 现存 7 等）。
- 文档维护负担：受检 23 个文档、4 个超限，最大 72 KB；策略是只提醒、不阻止、不回滚，故需人工消除。
- 新旧并行：旧采收器已停用且末次失败；演练残留目录数百个；多份旧版入口以备份形式留存。系统明确拒绝「靠删文件来收敛」，代价是读者须自行分辨哪些是当前权威。

10 缺陷的结构性的归因

收敛为四类根因。

根因一：把确定性职责交给概率性组件

维护、提炼、机器复核都要求模型产出可解析的结构。失败模式不是「偶尔慢」，而是整轮归零。

根因二：写入产能远大于消费产能

写入端持续高速产生，消费端（去重、校验、机器复核、人工策展、发布）每步收窄且末端必须人工。结果：66 条待审、193 条重复、约 1800 个来源积压、正式晋升 0。

根因三：观察 / 反馈闭环依赖一个已被自身数据证伪的机制

瓶颈不在「注入」，而在「回报」。回报依赖会话 Agent 中途自觉——而实测此项执行 0 次。

根因四：没有机制校验「文档承诺的能力是否仍存在」（影响最广）

系统有很强的文档纪律，却不知道自己的哪些部分已经失效：升权机制退役而文案仍生效、五级等级无实现、日报断更而导航仍列为现行、两套索引并存的说明未收敛。

前三条是「某个环节没做到」，这一条是「系统对自身状态失去认知」——对一个以记忆可信度立身的系统，这是最根本的风险。

11 总结评价

11.1 做对了什么

1. 把「会话起始」作为唯一的可靠注入点，并据此重建了整个架构。这是从实测推导出的设计，而非从良好愿望推导。
2. 权威 / 候选 / 派生 / 证据严格分档，未知不冒充已知，历史不补写证据，检测失败必须暴露。
3. 写入有版本、有登记、有幂等、有冲突拒绝——交接因此可以真正跨窗口、跨 Agent 可靠传递。
4. token 成本有多重硬约束（启动包 1800、热记忆 2 KB、记忆包 8 KB、检索 5 条），且采用「显式报告超支」而非「静默偷工」。
5. 对自身能力边界异常诚实——不谎报硬约束、不宣称已恢复完整状态、明确「不是安全沙箱」、真实接力失败时不伪装成功。
6. 设计意图层面提出了跨 Agent 独立验证、观察期分级、作用域多维模型等一系列有价值的机制构想——即使部分尚未落地，其思路本身仍有参考价值。

11.2 真实的短板

1. 多条被文档承诺的核心机制在实现层并不存在，而承诺它们的文案仍在生效——它侵蚀的是系统对自身状态的认知能力。
2. 自动化沉淀链押在本地模型的结构化输出上，失败即整轮归零；且存在两条从未缝合的候选来源，正式晋升计数始终为 0。
3. 写入产能远大于消费产能，积压是结构性的而非偶发的。
4. 观察 / 反馈闭环自我矛盾：依赖一个已被自身数据证伪的机制，导致 98% 冷反馈、1% 生效率、零强信号。
5. 数据安全双重缺口：状态层在仓库外且备份未核实，叠加版本控制长期未收口、快照目录为空。
6. 检索无语义能力、无真正增量，且边界声明不一致（部分是规则文本、无代码强制；内部告警被吞）。
7. 多宿主协作在测试里成立、在真机上未被验证（真实接力从未成功过一次）。
8. 一致性成本被有意转嫁给人工，设计上诚实，运营上是长期负担。

11.3 一句话概括

这套系统在「如何让多个 AI 共享可靠记忆」这个问题上，工程化程度相当高——分层、契约、版本、幂等、审计、预算、脱敏、能力边界声明一应俱全，且对自身局限的诚实程度远超同类系统。

但它面临两个层次的困难。运行层的困难在闭环末端：从「材料入库」到「经验可用」之间，横着一条需要模型能力与人工裁决的窄通道，目前这条通道的吞吐远低于写入端的产能，且中间还有一条从未缝合的断链。

认知层的困难更深：系统不知道自己的哪些部分已经失效。若干部件被退役或从未实现，而承诺它们的文案、规则和导航仍在生效。对一个以「记忆可信度」为立身之本的系统来说，这比任何单点技术故障都更根本。

换句话说：这套系统已经学会了如何记住，也学会了如何诚实地承认失败；但它还没有学会如何确认「自己声称拥有的能力是否仍然存在」。

附录：数据来源

数据类别	来源类型
启动包预算、档位与裁剪规则	工具层源码中的常量与裁剪分支
分层体积上限、注入规则	正式规则文件与派生记忆包
生效 / 观察档计数、冷反馈比、积压量	系统自身的健康度量产物
停用、失败、提炼归零、吞吐量	定时作业运行日志与状态文件
检索索引规模、并行旧索引	巡检结论与运维文档
状态层在仓库外、备份风险、快照为空	运维与数据保留文档的自述
Hook 五项检查与硬软分级	Hook 实现说明与测试用例
中途自觉执行 0 次	接入指南中的实测记录
定时作业返回码、端点鉴权状态	本机现场实测
机制在实现层是否存在、预算护栏有无调用点、晋升事件计数、索引是否增量	工具层源码与账本的逐项代码核查（对文档承诺做交叉验证）
未提交变更数、索引体积、分区一致性	现场只读清点

脱敏声明：本文已剔除全部本机绝对路径、用户名、设备标识、具体项目名称与业务细节。所有比例、计数、体积、预算数字均为系统自身产物的原值，未作估算或修饰。